

```
In [1]: # pip install nltk
```

```
In [2]: import nltk
from nltk.tokenize import word_tokenize
from nltk.tokenize import sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import string
```

```
In [3]: import nltk
import pandas as pd
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Pradnya\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\Pradnya\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
Out[3]: True
```

```
In [4]: df = pd.read_csv(r"Tweets.csv")
df.head()
```

```
Out[4]:
```

	textID	text	selected_text	sentiment
0	cb774db0d1	I'd have responded, if I were going	I'd have responded, if I were going	neutral
1	549e992a42	Sooo SAD I will miss you here in San Diego!!!	Sooo SAD	negative
2	088c60f138	my boss is bullying me...	bullying me	negative
3	9642c003ef	what interview! leave me alone	leave me alone	negative
4	358bd9e861	Sons of ****, why couldn't they put them on t...	Sons of ****,	negative

```
In [5]: #convert into lower case
df['text'] = df["text"].str.lower()
df['text']
df.info() # datatype-object
# Convert 'text' column to string data type
df['text'] = df['text'].astype(str)
type(df['text'][0])
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27481 entries, 0 to 27480
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   textID           27481 non-null  object
1   text             27480 non-null  object
2   selected_text    27480 non-null  object
3   sentiment        27481 non-null  object
dtypes: object(4)
memory usage: 858.9+ KB

```

Out[5]: str

```

In [6]: # Let's first split up the above text into sentences using sent_tokenize()
from nltk.tokenize import sent_tokenize
from nltk.probability import FreqDist

text = "Hello . its sakshi nakate. but you can call me Gulaboo. I am from Kumbho
text_to_sentence = sent_tokenize(text)
print(type(text_to_sentence))
freq_dist_of_words = FreqDist(text_to_sentence)
freq_dist_of_words
freq_dist_of_words.most_common(5)

```

```

<class 'list'>

```

```

Out[6]: [('Hello .', 1),
        ('its sakshi nakate.', 1),
        ('but you can call me Gulaboo.', 1),
        ('I am from Kumbhoj and My friend lives in vita.', 1),
        ('She didn't invite me on Eid.', 1)]

```

```

In [7]: df['sent_token'] = df['text'].apply(nltk.sent_tokenize)
# df['sent_token'] = df['text'].apply(nltk.word_tokenize)

```

```

In [8]: df['sent_token'].head()

```

```

Out[8]: 0      [ i`d have responded, if i were going]
1      [ sooo sad i will miss you here in san diego!!...
2      [my boss is bullying me...]
3      [ what interview!, leave me alone]
4      [ sons of ****, why couldn`t they put them on ...
Name: sent_token, dtype: object

```

```

In [9]: df['word_token'] = df['text'].apply(nltk.word_tokenize)

```

```

In [10]: df['word_token'].head()

```

```

Out[10]: 0      [i, `, d, have, responded, ,, if, i, were, going]
1      [sooo, sad, i, will, miss, you, here, in, san,...
2      [my, boss, is, bullying, me, ...]
3      [what, interview, !, leave, me, alone]
4      [sons, of, *, *, *, *, ,, why, couldn, `, t, t...
Name: word_token, dtype: object

```

```

In [ ]: from nltk.probability import FreqDist
data = df.copy()
# data

```

```

for i in data['word_token']:
    freq_dist_of_words = FreqDist(i)
    freq_dist_of_words
    print(freq_dist_of_words.most_common(5))

```

```

In [12]: # Removing Stop words:
# nltk.download('stopwords')
from nltk.corpus import stopwords

stopwords = nltk.corpus.stopwords.words('english')
stopwords
len(stopwords)
print(list(stopwords))

```

```

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'a
nd', 'any', 'are', 'aren', "aren't", 'as', 'at', 'be', 'because', 'been', 'befor
e', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', "could
n't", 'd', 'did', 'didn', "didn't", 'do', 'does', 'doesn', "doesn't", 'doing', 'd
on', "don't", 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had',
'hadn', "hadn't", 'has', 'hasn', "hasn't", 'have', 'haven', "haven't", 'having',
'he', "he'd", "he'll", 'her', 'here', 'hers', 'herself', "he's", 'him', 'himsel
f', 'his', 'how', 'i', "i'd", 'if', "i'll", "i'm", 'in', 'into', 'is', 'isn', "is
n't", 'it', "it'd", "it'll", "it's", 'its', 'itself', "i've", 'just', 'll', 'm',
'ma', 'me', 'mightn', "mightn't", 'more', 'most', 'mustn', "mustn't", 'my', 'myse
lf', 'needn', "needn't", 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'onc
e', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 'r
e', 's', 'same', 'shan', "shan't", 'she', "she'd", "she'll", "she's", 'should',
'shouldn', "shouldn't", "should've", 'so', 'some', 'such', 't', 'than', 'that',
"that'll", 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'thes
e', 'they', "they'd", "they'll", "they're", "they've", 'this', 'those', 'throug
h', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'wasn', "wasn't",
'we', "we'd", "we'll", "we're", 'were', 'weren', "weren't", "we've", 'what', 'whe
n', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', "wo
n't", 'wouldn', "wouldn't", 'y', 'you', "you'd", "you'll", 'your', "you're", 'you
rs', 'yourself', 'yourselves', "you've"]

```

```

In [13]: # # Remove stopwords

# # -----

# Sample DataFrame
# data = {
#     'tokens': [['the', 'cat', 'is', 'on', 'the', 'mat'],
#                ['the', 'dog', 'barked', 'loudly'],
#                ['she', 'is', 'a', 'good', 'friend']]
# }

# data = pd.DataFrame(data)

# # List of stopwords
# stopwords = ['the', 'is', 'a', 'on']

# # Filtering out stopwords
# type(data['tokens'])
# data['word_tokens'] = data['tokens'].apply(lambda x: [word for word in x if wo

```

```
# print(data['tokens'])
# # -----

df['word_token'] = df['word_token'].apply(lambda x: [word for word in x if word
df['word_token']
# df
```

```
Out[13]: 0          [, responded, ,, going]
1          [sooo, sad, miss, san, diego, !, !, !]
2          [boss, bullying, ...]
3          [interview, !, leave, alone]
4          [sons, *, *, *, *, ,, `, put, releases, ahead...
          ...
27476      [wish, could, come, see, u, denver, husband, l...
27477      [, wondered, rake, ., client, made, clear, .n...
27478      [yay, good, ., enjoy, break, -, probably, need...
27479          [worth, *, *, *, *, .]
27480      [flirting, going, -, atg, smiles, ., yay, ., (...
Name: word_token, Length: 27481, dtype: object
```

```
In [14]: # (d) Stemming:
# Stemming is a process of normalization, in which words are reduced to their ro
from nltk.stem.snowball import SnowballStemmer
import nltk.stem.snowball
snow_stem = SnowballStemmer(language='english')
# -----
# words = ['happy', 'happier', 'happiest', 'happiness', 'breathing', 'fairly', 'ea
# for word in words:
#     print(word + '--->' + snow_stem.stem(word))
# -----
# df['word_token'] = df['word_token'].apply(snow_stem.stem) ----steam works on s
# Apply stemming to each word in the lists
df['word_token'] = df['word_token'].apply(lambda x : [snow_stem.stem(word) for w
df['word_token'][1]
# df
```

```
Out[14]: ['sooo', 'sad', 'miss', 'san', 'diego', '!', '!', '!']
```

```
In [15]: # Like stemming, Lemmatization is also used to reduce the word to their root wor
# It uses vocabulary and morphological analysis to transform a word into a root
nltk.download('wordnet')
from nltk.stem.wordnet import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

df['word_token'] = df['word_token'].apply(lambda x:[lemmatizer.lemmatize(word) f
df['word_token']
```

```
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\Pradnya\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
```

```

Out[15]: 0          [, respond, ,, go]
1          [sooo, sad, miss, san, diego, !, !, !]
2          [bos, bulli, ...]
3          [interview, !, leav, alon]
4          [son, *, *, *, *, ,, `, put, releas, already, ...
          ...
27476      [wish, could, come, see, u, denver, husband, l...
27477      [, wonder, rake, ., client, made, clear, .net...
27478      [yay, good, ., enjoy, break, -, probabl, need,...
27479          [worth, *, *, *, *, .]
27480      [flirt, go, -, atg, smile, ., yay, ., (, (, hu...
Name: word_token, Length: 27481, dtype: object

```

```

In [16]: from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score

```

```

In [17]: # now Split the dataset into training and testing sets
df.columns
# df.drop('textID',axis=1)

X = df['text']
y = df['sentiment']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

```

```

In [18]: vectorizer = TfidfVectorizer()
X_train_vectors = vectorizer.fit_transform(X_train)
X_test_vectors = vectorizer.transform(X_test)

```

```

In [19]: # choose a machine Learning algorithm, such as Support Vector Machines (SVM),
# and train it on the training data

```

```

In [20]: model = SVC(kernel='linear')
model.fit(X_train_vectors, y_train)

```

```

Out[20]: SVC
SVC(kernel='linear')

```

```

In [21]: y_pred = model.predict(X_test_vectors)

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Accuracy Score:", accuracy_score(y_test, y_pred))

```

Classification Report:

	precision	recall	f1-score	support
negative	0.74	0.63	0.68	1562
neutral	0.65	0.77	0.70	2230
positive	0.80	0.72	0.76	1705
accuracy			0.71	5497
macro avg	0.73	0.71	0.71	5497
weighted avg	0.72	0.71	0.71	5497

Accuracy Score: 0.712934327815172

```
In [27]: # Correcting the prediction line
input_text1 = ["Heyy you idiot!"] # Input must be a list
input_text2 = ["Hello preety grl!"] # Input must be a list
input_text3 = ["Loved your car!"] # Input must be a list
# Making the prediction

def checkSentiments(str):
    input_vector = vectorizer.transform(str) # Vectorizing the input
    print(model.predict(input_vector))

checkSentiments(input_text1)
checkSentiments(input_text2)
checkSentiments(input_text3)
```

```
['negative']
['neutral']
['positive']
```